

数理逻辑课程大作业实验报告

李政轩 续博森 吴韬 陈怡琼 许浩博

2026 年 7 月 18 日

摘要

本文报告围绕 NesyLink 数理逻辑课程项目展开，目标是在 Zelda-like 地牢环境中设计可运行的 Agent，并对环境或策略中的关键部分进行 Lean 形式化建模与证明。

本项目已对 `task_1` 至 `task_5` 全部完成 Lean 环境形式化与可完成性证明，对应文件位于 `student_agent/lean/` 目录下。五个形式化模型均通过 Lean LSP 诊断（零错误），并遵循统一证明骨架：局部前置/后置引理 $\rightarrow$ 必要性引理 $\rightarrow$ `Reachable` 归纳不变式 $\rightarrow$ `completion_postcondition` $\rightarrow$ witness 计划 $\rightarrow$ 可完成性。其中 `task_4` 因涉及旋转桥、条件门、装备获取与事件驱动显现机制，作为重点展开对象。此外，本文对 `task_1` 至 `task_5` 进一步补充了高层符号策略形式化：`task_1` 至 `task_4` 采用 BFS 证书模式（`symbolicPolicy` 参数化于 BFS 动作提供器，通过证书结构证明非固定路线完备性），`task_5` 在房间级阶段机之上增加与最终 Python baseline 对齐的证书细化层：10 类 BFS 路线、两类战斗契约、西门举盾入场契约和 24 个宏动作阶段均可计算检查，并证明五个任务的符号策略从初态出发均能选择合法动作且到达对应目标。在此基础上，本文进一步抽取了独立的 `StrategyFramework.lean`，形式化通用策略执行、动作 mask 安全证书、有界可达性和 BFS 访问集，并证明“若某目标在 n 步内由某个计划可达，则完整动作集上的深度 n BFS 定会访问到目标态”。

目录

1 项目背景与任务要求	4
1.1 课程作业目标	4
1.2 环境限制与评测要求	4
1.3 本文结构	4
2 环境与任务分析	4
2.1 NesyLink 环境概述	4
2.2 任务拆解	5

目录	2
2.3 主要难点	5
3 总体方法设计	5
3.1 系统框架	5
3.2 为什么选择该路线	6
4 视觉与状态抽取	6
4.1 输入与输出设计	6
4.2 实现方法	7
4.3 误差与局限	7
5 策略与规划	7
5.1 基线策略	7
5.2 多任务扩展	8
5.3 最终提交版本	8
6 Lean 形式化与证明	9
6.1 形式化对象总览	9
6.2 Task 1: 单房间钥匙门形式化	9
6.3 Task 2: 战斗与条件门形式化	11
6.4 Task 3: 多房间往返形式化	12
6.5 Task 4: 旋转桥与隐藏宝箱形式化	13
6.6 Task 5: 四房间按钮门与全宝箱收集形式化	15
6.6.1 环境建模	15
6.6.2 状态空间与转移函数	16
6.6.3 核心定理	16
6.6.4 归纳不变式	17
6.6.5 Witness 计划	18
6.6.6 策略形式化与正确性证明	18
6.7 统一策略框架与有界完备性证明	21
6.8 证明统一思路	22
6.9 证明局限与边界	23
7 实验设计与结果分析	23
7.1 实验设置	23
7.2 实验结果	25
7.3 结果分析	26
7.3.1 Task 5 状态说明	27
7.3.2 颜色变体鲁棒性修复	30

目录	3
8 项目分工与推进过程	32
8.1 成员分工	32
8.2 时间安排	33
9 总结与展望	36
A 运行证明	37
A.1 Lean 形式化验证	37
A.2 Robustness suite 评测运行	37

1 项目背景与任务要求

1.1 课程作业目标

本次课程作业要求小组围绕给定游戏环境完成三部分内容：

1. 设计并实现一个能够完成任务的 Agent；
2. 使用 Lean 对环境或策略中的关键部分进行形式化建模；
3. 证明若干性质，例如动作合法性、安全性、可达性或策略正确性。

1.2 环境限制与评测要求

- 训练和调试阶段可以使用像素、reward 和 info 等信息。
- 最终测试阶段，Agent 不能直接依赖环境内部 info。
- 最终测试阶段，Agent 只能根据像素观测、reward 和物品栏信息进行决策。
- 如果使用机器学习方法，需要说明训练方式、信息来源以及与最终评测接口的一致性。

1.3 本文结构

本文后续将依次介绍环境与任务分析、总体方法设计、视觉与状态抽取、策略与规划、Lean 形式化与证明（覆盖 task_1 至 task_5）、实验结果以及项目分工。

2 环境与任务分析

2.1 NesyLink 环境概述

NesyLink 是一个 Zelda-like 的离散动作地牢环境。默认任务接口采用 Gym 风格：`obs, info = env.reset(seed)`，以及 `obs, reward, terminated, truncated, info = env.step(action)`。本次课程任务要求最终提交版策略在推理阶段只能使用像素观测、reward 和显式提供的物品栏信息，不能直接读取 info 中的隐藏状态。

动作空间为 7 个离散动作：WAIT、UP、DOWN、LEFT、RIGHT、BUTTON_A、BUTTON_B。默认像素控制下，房间地图区域大小为 160×128 像素，对应 10×8 个 tile，每个 tile 为 16×16 像素。默认观测 obs 为 RGB 图像，shape 为 (128, 160, 3)。环境还提供结构化 info 作为调试与监督接口，其中包含房间编号、角色坐标、事件记录、动态对象状态和奖励信号等信息。

五个任务的差异主要体现在关键机制上：`task_1` 是静态单房间取钥匙开门；`task_2` 引入怪物、战斗和条件门；`task_3` 要求跨房间记忆；`task_4` 引入旋转桥、条件门、装备获取和隐藏宝箱，是最适合做显式环境建模的一关；`task_5` 则是四房间按钮门与全宝箱收集的综合任务。基于课程要求与当前项目进度，本文对 `task_1` 至 `task_5` 全部进行了 Lean 环境形式化与可完成性证明，其中 `task_4` 因机制最复杂作为重点展开对象。

2.2 任务拆解

五个任务可以按关键机制逐步拆解：

- Task 1: 单房间取钥匙开门。
- Task 2: 怪物处理与取钥匙。
- Task 3: 多房间状态记忆与往返。
- Task 4: 桥与按钮机制、复杂子任务链。
- Task 5: 综合关卡与泛化挑战。

2.3 主要难点

1. 最终测试不能直接读取 `info`。
2. 需要从像素中抽取对策略有用的状态表示。
3. 需要在短时间内兼顾实现、证明和报告。

3 总体方法设计

3.1 系统框架

本项目采用“像素感知 + 规则状态机 + Lean 符号层验证”的整体框架。运行时，`student_agent/baseline_policy.py` 提供统一的 Policy 入口，评测脚本仍按 `act(obs, info)` 调用，但策略主体尽量只使用像素观测、reward 信号、物品栏和内部历史记忆。整体链路可以分为四个模块：

- 视觉/状态抽取模块：从 `obs` 中识别玩家中心 tile、宝箱、NPC、怪物、开关、按钮、墙体、桥方向和房间特征。
- 策略与规划模块：按任务维护阶段记忆，将任务拆成“拿钥匙”“按按钮”“切桥”“开宝箱”“击杀怪物”“进入出口”等子目标。

- **反馈记忆模块:**在最终测评的 safe 信息模式下,info 仅包含 last_reward、inventory 和 task_id,不再暴露 reward_signals。策略通过物品栏 diff (keys/gold/items 增量)结合 last_reward 和像素感知(怪物消失、按钮颜色变化)推断 key_collected、chest_opened、monster_killed、button_pressed 等关键事件,避免依赖隐藏事件列表。
- **形式化验证模块:**在 Lean 中抽象环境的符号层语义,证明关键前置条件、不变式、完成后置条件、witness 计划可完成性,并对 task_5 的高层阶段机策略给出动作合法性与目标可达性证明。

3.2 为什么选择该路线

选择该路线主要基于三点考虑。

- **可解释性:**五个任务均有清晰的目标链,规则阶段机比黑盒模型更容易定位失败原因。
- **与 Lean 对齐:**Lean 证明适合覆盖钥匙门、按钮门、桥状态、宝箱开启、怪物击杀和完成谓词这类符号语义,和阶段机策略天然一致。
- **实现风险可控:**课程时间有限,先做可运行 baseline,再把高层环境语义形式化,比从头训练复杂策略更稳健。

当前阶段已经选择“统一 Policy 骨架 + 分任务逐步扩展 + 可验证层优先”的路线。具体而言,项目先在自有代码目录中建立统一的 Agent 入口,优先跑通 task_1 的 baseline,再逐步扩展到 task_2 至 task_5,并以阶段机、合法动作筛选器和动态对象状态跟踪作为后续形式化重点。目前自有 baseline 已覆盖 task_1 至 task_5。其中 task_2 仍属于调试用途的脚本化版本,而 task_3、task_4 和 task_5 已经采用像素房间识别、玩家中心格定位、物品栏和 reward 历史驱动的规则状态机。Lean 部分首先完成五个任务的环境形式化与 witness 可完成性证明;在此基础上,task_5 还额外形式化了与 Python 阶段机对应的高层符号策略,证明策略生成的房间级动作序列合法,并能达到全宝箱开启目标。

4 视觉与状态抽取

4.1 输入与输出设计

视觉模块输入为 RGB 像素帧 obs。输出不是完整地图重建,而是服务于规则策略的少量符号状态:

- **玩家中心 tile:**根据玩家绿色精灵像素反推精灵左上角,再取中心点所在 tile。

- 关键对象：通过颜色计数识别宝箱、NPC、怪物、墙体、开关、按钮、深渊和桥。
- 房间类别：针对 `task_3`、`task_4`、`task_5` 分别用房间内稳定对象和地形特征识别当前房间。
- 动态状态：`task_4` 通过桥像素分布识别 `west_to_north`、`west_to_east`、`west_to_south` 三种桥方向；`task_5` 通过按钮、宝箱和房间特征识别当前子任务位置。

4.2 实现方法

实现上采用基于颜色模板和 tile 统计的轻量规则方法。`baseline_policy.py` 中定义了玩家、墙、宝箱、NPC、怪物、桥、开关和按钮的颜色常量，并提供 `count_color`、`count_any_color`、`tile_has_chest`、`tile_has_switch`、`tile_has_button` 等辅助函数。

- 玩家定位不直接读取 `info["agent"]`，而是从像素中恢复中心 tile。
- 房间识别不直接读取 `info["env"]`，而是通过房间内宝箱、NPC、桥、开关、墙体开口等稳定视觉特征判断。
- 怪物、按钮、开关和宝箱状态通过像素与物品栏/`last_reward` 共同维护；例如 `task_5` 用 `inventory diff (keys/gold 增量)`、`last_reward` 正值和怪物像素消失来推断 `button_pressed`、`chest_opened`、`monster_killed` 等事件并更新内部记忆。
- 物品栏只用于读取课程允许的钥匙、装备等显式状态。

4.3 误差与局限

该视觉方案依赖当前地图素材的固定颜色和固定 tile 尺寸，因此更适合作为课程环境内的可解释 baseline，而不是通用视觉模型。主要风险包括：角色精灵中心与碰撞箱位置存在偏差、tile 已到位但碰撞箱仍未越过出口边界、以及对象被开启或按下后像素外观变化导致房间识别失败。针对这些问题，策略中加入了若干行/列对齐阶段，例如 `task_5` 进入东门、从东房返回、进入西房前都保留额外对齐动作，以减少碰撞边界误差。

5 策略与规划

5.1 基线策略

基线策略采用规则式阶段机 + BFS 路径规划，而不是学习模型。

- `task_1`: BFS 到宝箱邻居 → 开箱取钥匙 → BFS 到北门出口。

- `task_2`: 像素感知怪物 → 邻接攻击 → BFS 到宝箱 → 开箱 → BFS 到西出口。
- `task_3`: BFS 跨房间西行到钥匙房 → 开箱 → BFS 返回起点 → 开锁门。
- `task_4`: 维护桥方向和阶段记忆, 按“北房钥匙 → 切桥到东房拿剑 → 切桥到南房杀守卫 → 回中心开最终宝箱”推进, 房间内导航用 BFS。
- `task_5`: 维护按钮、钥匙、东门、怪物、四个宝箱等事件记忆, 按“起始宝箱 → 按按钮 → 南房钥匙 → 起始房杀怪 → 东房回血 → 西房金币”推进, 事件通过 `inventory diff + last_reward + 像素感知推断`。

5.2 多任务扩展

多任务扩展的核心是把每个任务拆成少量稳定子目标, 并在 `AgentHistory.notes` 中保存跨步记忆。对单房间任务, 固定计划即可完成; 对多房间任务, 策略需要记录当前阶段、是否已开箱、是否已取得钥匙、桥是否已旋转到目标方向、怪物是否已经击杀等信息。路径选择主要使用 tile 级贪心移动和固定 waypoint, 在出口和交互点附近加入额外对齐动作, 保证像素控制下的碰撞箱能够真正进入交互范围。

5.3 最终提交版本

当前提交版本与调试过程的区别如下:

- 调试阶段曾使用 `info` 校准房间、坐标和事件语义, 便于定位失败原因。
- 当前 baseline 已全面适配评测脚本的 `safe` 信息模式:`reset()` 不再接收 `seed/task_id` 参数, `task_id` 改由 `info["task_id"]` 获取 (通过 `--task-policy` 绑定); 原先依赖 `reward_signals` 的 `update_memory` 与 `update_task5_memory` 已重构为基于 `inventory diff`、`last_reward` 和像素感知的事件推断。
- 五个任务的主体决策均采用像素识别 + BFS 路径规划 + 阶段机, 不直接读取隐藏的 `info["env"]`、`info["agent"]`、`info["dynamic"]` 或 `info["events"]`。
- Lean 验证了五个任务的符号层环境、显式 witness 计划, 以及全部五个任务的高层符号策略 (`symbolicPolicy`) 的合法性与完成性——`task_1` 至 `task_4` 采用 BFS 证书模式 (策略参数化于 BFS 动作提供器, 证书结构证明非固定路线完备性), `task_5` 在原有 `PolicyStage` 房间级证明之上, 新增 BFS、战斗和西门举盾入场证书驱动的 24 阶段 baseline 细化; 同时新增通用策略框架文件, 证明有界 BFS 搜索的框架级完备性; 尚未证明像素感知层一定能在所有渲染扰动下恢复这些符号状态。

6 Lean 形式化与证明

6.1 形式化对象总览

本项目对 task_1 至 task_5 全部完成了 Lean 环境形式化与可完成性证明，对应文件位于 student_agent/lean/ 目录下：

- Task1Formalization.lean: tile 级单房间钥匙门；
- Task2Formalization.lean: zone 级战斗与条件门；
- Task3Formalization.lean: room 级三房间往返取钥匙；
- Task4Formalization.lean: room 级旋转桥与隐藏宝箱；
- Task5Formalization.lean: room 级四房间按钮门与全宝箱收集；
- StrategyFramework.lean: 通用策略执行、动作 mask 安全证书、有界可达性与 BFS 完备性证明。

任务形式化文件均通过 Lean LSP 诊断（零错误），并按照统一的设计原则建模：

1. **抽象层级与 Python 行为对齐：** 根据每个任务的关键机制选择 tile / zone / room 抽象层级，并与 movement.py、interactions.py、combat.py 及 room_*.json 中的语义保持一致；
2. **保留约束而不照搬实现：** 刻意忽略像素、碰撞盒、怪物 AI、击退等低层细节，只保留影响规划正确性的状态字段、前置条件与事件触发；
3. **统一证明骨架：** 每个任务都给出“局部前置/后置引理 $\rightarrow$ 必要性引理 $\rightarrow$ Reachable 归纳不变式 $\rightarrow$ completion_postcondition $\rightarrow$ witness 计划 $\rightarrow$ 可完成性”的完整证明链。

下面分任务展开。其中 task_4 因机制最复杂（动态对象状态、条件门、关键道具获取和事件驱动显现），作为重点展开对象。

6.2 Task 1: 单房间钥匙门形式化

task_1 是最简单的单房间取钥匙走北门任务。Lean 文件以 tile 级建模，对应 room_001.json 的 8 行 $\times$ 10 列网格。

状态与动作：

- EnvState: $x, y : \text{Nat}$ (tile 坐标)、keys、chestOpened、completed；
- Action: up / down / left / right / openChest / goNorth；

- `isWalkable` 严格复刻 `room_001.json` 中的墙体布局 (row 2 的 `##.#####`、row 5 的 `#####...`);
- `isAdjacentToChest` 使用 Manhattan 距离 ≤ 1 , 对齐 `state.py:is_adjacent` 的语义 (包含距离 0)。

关键约束:

- `openChest` 触发条件为邻接 chest tile 且未开过, 开箱后 `keys + 1`;
- `goNorth` 仅在 (4, 0) 且 `keys > 0` 时触发, 成功后消耗一把钥匙并将 `completed` 置真 (对应 `consume_key: true`)。

定理名称	形式化对象	结论说明
<code>step_preserves_walkable</code>	安全性	任意动作后玩家仍位于可走 tile 上。
<code>openChest_only_at_chest</code>	开箱前置条件	不邻接 chest 时 <code>openChest</code> 不改变状态。
<code>openChest_at_chest_increases_keys</code>	开箱后置条件	邻接且未开过时, <code>keys</code> 增加 1。
<code>goNorth_only_at_exit</code>	北门前置条件	不在 (4, 0) 时 <code>goNorth</code> 无效。
<code>goNorth_without_key</code>	北门前置条件	在出口但无钥匙时 <code>goNorth</code> 无效。
<code>goNorth_at_exit_with_key_completes</code>	北门后置条件	在出口且持钥匙时 <code>completed</code> 置真。
<code>only_goNorth_sets_completed</code>	必要性引理	<code>goNorth</code> 是唯一能把 <code>completed</code> 从假翻为真的动作。
<code>witnessPlan_reaches_goal</code>	执行轨迹	22 步 witness 计划可达目标。
<code>task1_completable</code>	可完成性	<code>task_1</code> 在该模型下可完成。

witness 计划: tile 级路径

$(4, 6) \xrightarrow{\text{right} \times 3} (7, 6) \xrightarrow{\text{up} \times 3} (7, 3) \xrightarrow{\text{left} \times 7} (0, 3) \xrightarrow{\text{openChest}} \cdot \xrightarrow{\text{right} \times 3, \text{up} \times 3, \text{right}} (4, 0) \xrightarrow{\text{goNorth}} \text{goal}.$

由于 22 步计划超出 `decide` 递归深度, 按项目约定使用 `native_decide` 验证以下定理:

`witnessPlan_executes_to_finalState`

6.3 Task 2: 战斗与条件门形式化

`task_2` 在单房间内引入 `chaser` 怪物、HP、陷阱与西向条件门。为突出战斗与条件门语义, 采用 `zone` 级抽象。

状态与动作:

- `Zone: safeCenter / nearMonster / nearChest / westExit / topTrap / bottomTrap;`
- `EnvState: zone、hp、monsterAlive、monsterHp、chestOpened、keys、completed;`
- `Action: moveTo z / attack / openChest / useExit / wait.`

关键约束:

- `attack` 仅在 `nearMonster`、怪物存活且 `hp > 1` 时生效; `monsterHp` 为 1 时击杀 (置 `monsterAlive := false`), 否则仅 `monsterHp - 1`;
- `useExit` 在 `westExit` 且怪物已死且 `keys > 0` 时触发, 成功后不消耗钥匙 (对应 `requires` 中未设 `consume_key`);
- 走入 `topTrap / bottomTrap` 时 HP 减 1 并被重置回 `safeCenter`。

定理名称	形式化对象	结论说明
<code>trap_move_reduces_hp</code>	陷阱语义	踩陷阱后 <code>hp</code> 减少 1。
<code>safe_move_preserves_*</code>	安全移动不变式	非陷阱 <code>moveTo</code> 保持 HP/怪物/宝箱/钥匙/完成状态不变。
<code>attack_reduces_monsterHp_when_adjacent</code>	战斗语义	邻接且存活且 HP 充足时, <code>monsterHp</code> 减 1。
<code>attack_kills_monster_when_monsterHp_one</code>	战斗语义	<code>monsterHp = 1</code> 时一击击杀。
<code>attack_no_effect_*</code>	战斗前置条件	不在 <code>nearMonster</code> 、怪物已死或 HP 过低时攻击无效。
<code>exit_blocked_if_monster_alive</code>	条件门	怪物存活时 <code>useExit</code> 无效。
<code>exit_blocked_without_key</code>	条件门	无钥匙时 <code>useExit</code> 无效。
<code>exit_succeeds_after_requirements</code>	条件门后置	怪物已死且持钥匙时 <code>completed</code> 置真。

定理名称	形式化对象	结论说明
<code>only_useExit_sets_completed</code>	必要性引理	<code>useExit</code> 是唯一完成动作。
<code>completion_postcondition</code>	完成后置	任意可达完成态满足怪物已死 $\wedge$ 宝箱已开。
<code>hp_non_increasing</code>	安全性	任意动作后 HP 不增。
<code>witnessPlan_reaches_goal</code>	执行轨迹	7 步 witness 计划可达目标。
<code>task2_completable</code>	可完成性	<code>task_2</code> 在该模型下可完成。

witness 计划： zone 级路径

$\text{safeCenter} \rightarrow \text{nearMonster} \xrightarrow{\text{attack} \times 2} \cdot \rightarrow \text{nearChest} \xrightarrow{\text{openChest}} \cdot \rightarrow \text{westExit} \xrightarrow{\text{useExit}} \text{goal}.$

6.4 Task 3: 多房间往返形式化

`task_3` 是首个多房间任务：从 `start_room` 出发向西穿过 `monster_hall` 到达 `key_room`，取钥匙后返回 `start_room` 走东向锁定出口。采用 room 级抽象。

状态与动作：

- Room: `startRoom` / `monsterHall` / `keyRoom`;
- EnvState: `room`、`keys`、`keyChestOpened`、`hallMonsterAlive`、`completed`;
- Action: `goWest` / `goEast` / `openChest` / `openLockedExit` / `wait`。

关键约束：

- `goWest: startRoom  $\rightarrow$  monsterHall  $\rightarrow$  keyRoom`; `goEast`: 反向; `keyRoom` 的 `goWest` 与 `startRoom` 的 `goEast` 无效;
- `openChest` 仅在 `keyRoom` 且未开过时生效，开箱后 `keys + 1`;
- `openLockedExit` 仅在 `startRoom` 且 `keys > 0` 时触发，成功后消耗钥匙并完成（对应 `consume_key: true`）。

定理名称	形式化对象	结论说明
<code>west_then_west_reaches_keyRoom</code>	导航语义	从 <code>startRoom</code> 连续两次 <code>goWest</code> 可达 <code>keyRoom</code> 。

定理名称	形式化对象	结论说明
<code>openChest_only_in_keyRoom</code>	开箱前置	不在 <code>keyRoom</code> 时 <code>openChest</code> 无效。
<code>openChest_in_keyRoom_increases_keys</code>	开箱后置	在 <code>keyRoom</code> 且未开过时 <code>keys</code> 增 1。
<code>openLockedExit_blocked_without_key</code>	锁门前置	在 <code>startRoom</code> 但无钥匙时 <code>openLockedExit</code> 无效。
<code>openLockedExit_consumes_key_and_completes</code>	锁门后置	持钥匙时消耗一把钥匙并完成。
<code>hallMonsterAlive_never_changes</code>	不变式	Task 3 无攻击动作, <code>hallMonsterAlive</code> 恒不变。
<code>only_openLockedExit_sets_completed</code>	必要性引理	<code>openLockedExit</code> 是唯一完成动作。
<code>keyChestOpened_or_keys_zero</code>	归纳不变式	任意可达状态: 宝箱已开 $\vee$ 钥匙数为 0。
<code>completion_postcondition</code>	完成后置	任意可达完成态满足 <code>keyChestOpened = true</code> 。
<code>witnessPlan_reaches_goal</code>	执行轨迹	6 步 <code>witness</code> 计划可达目标。
<code>task3_completable</code>	可完成性	<code>task_3</code> 在该模型下可完成。

witness 计划:

$$\text{startRoom} \xrightarrow{\text{goWest} \times 2} \text{keyRoom} \xrightarrow{\text{openChest}} \cdot \xrightarrow{\text{goEast} \times 2} \text{startRoom} \xrightarrow{\text{openLockedExit}} \text{goal}.$$

由于计划简单, `witnessPlan_executes_to_finalState` 用 `rfl` 直接验证。

6.5 Task 4: 旋转桥与隐藏宝箱形式化

`task_4` 引入旋转桥、条件门、装备获取与事件驱动显现机制, 是机制最复杂的一关, 也是本报告的重点展开对象。对应 Lean 文件为 `Task4Formalization.lean`, 与 Python 环境中以下模块对齐: `movement.py` (门与房间切换)、`interactions.py` (开关与宝箱)、`combat.py` (击杀怪物后揭示宝箱)。

具体抽象:

- 房间状态: 使用 `Room` 枚举描述五个房间 `west / center / north / east / south`。
- 动态对象状态: 使用 `BridgeState` 描述中央桥的三种连通状态 `westToNorth / westToEast / westToSouth`。

- **玩家资源与关键事件：**记录 `keys`、`hasSword`、`hasShield`，以及北房宝箱、东房宝箱、南房守卫和中央终点宝箱的状态。
- **动作：**抽象为房间级动作，包括转桥、进出中心房、开宝箱、攻击和开启最终宝箱。
- **转移函数：**用确定性函数 `step` 描述状态更新。它保留了三类关键约束：东门需要钥匙但不消耗钥匙（对应 `consume_key: false`）；桥状态决定中心房可达方向；南房守卫被击杀后才会揭示中央最终宝箱。
- **目标谓词：**`GoalReached` 定义为最终宝箱已开启。

定理名称	形式化对象	结论说明
<code>rotateBridge_three_cycle</code>	桥状态机	证明西房开关连续触发三次后，桥状态回到初始方向。
<code>goEast_blocked_without_key</code>	东门条件	证明当玩家位于中心房、桥已连到东侧但没有钥匙时， <code>goEast</code> 不产生状态变化。
<code>goEast_succeeds_with_key</code>	东门条件	证明在中心房、桥连东侧且已持钥匙时， <code>goEast</code> 能把玩家带入东房。
<code>goEast_preserves_keys</code>	东门不变式	证明 <code>goEast</code> 不消耗钥匙（Python <code>consume_key: false</code> 对齐）。
<code>attack_without_sword_has_no_effect</code>	战斗语义	证明没有剑时，在南房执行攻击不会击杀守卫，也不会揭示最终宝箱。
<code>attack_with_sword_reveals_final_chest</code>	战斗与事件触发	证明持剑击败南房守卫后， <code>guardianAlive</code> 变为假，且 <code>finalChestVisible</code> 变为真。
<code>openFinalChest_requires_visibility</code>	终点交互语义	证明若最终宝箱尚未显现，则在中心房尝试开启最终宝箱不会成功。
<code>only_openFinalChest_sets_finalChestOpened</code>	必要性引理	<code>openFinalChest</code> 是唯一完成动作。
<code>northChestOpened_or_keys_zero</code>	归纳不变式	任意可达状态：北房宝箱已开 $\vee$ 钥匙数为 0。

定理名称	形式化对象	结论说明
<code>finalChestVisible_</code> <code>implies_guardianDead</code> <code>completion_</code> <code>postcondition</code>	归纳不变式 完成后置	任意可达状态：终点宝箱显现 ⇒ 守卫已死。 任意可达完成态满足 <code>guardianAlive = false</code> 。
<code>witnessPlan_reaches_</code> <code>goal</code>	执行轨迹	给出一条具体计划，证明从初始状态出发可以完成“取钥匙 → 拿剑 → 击杀守卫 → 开最终宝箱”的任务链。
<code>task4_completable</code>	可达性	由上述执行轨迹推出：在该形式化模型下， <code>task_4</code> 是可完成的。

witness 计划：17 步房间级路径

$$\begin{aligned}
 &\text{west} \xrightarrow{\text{goCenter, goNorth, openChest}} \text{north} \xrightarrow{\text{goCenter, goWest, toggleBridge}} \text{west} \\
 &\quad \xrightarrow{\text{goCenter, goEast, openChest}} \text{east} \xrightarrow{\text{goCenter, goWest, toggleBridge}} \text{west} \\
 &\quad \xrightarrow{\text{goCenter, goSouth, attack}} \text{south} \xrightarrow{\text{goCenter, openFinalChest}} \text{goal}.
 \end{aligned}$$

`witnessPlan_executes_to_finalState` 用 `rfl` 直接验证。

6.6 Task 5：四房间按钮门与全宝箱收集形式化

6.6.1 环境建模

Task 5 包含 4 个房间 (`startRoom` / `southRoom` / `eastRoom` / `westRoom`)，关键机制为：

- 按钮门：`startRoom` 的南出口为条件门 (`button_pressed`)，初始 `buttonPressed = false`；
- 钥匙门：`startRoom` 的东出口为锁门 (`consume_key = true`)，初始 `keys = 0`；
- 差异化宝箱：4 个房间各有 1 个宝箱，效果不同——`startRoom` (金币，无副作用)、`southRoom` (钥匙，`keys + 1`)、`eastRoom` (回血，`hp + 1`)、`westRoom` (金币，无副作用)；
- 世界完成：与 `task_1--4` 不同，`task_5` 无 `complete_task` 出口，世界完成条件为 `all_chests_opened` (对齐 `progress.py:36`)。

GoalReached 定义为 allChestsOpened, 即四个宝箱全部打开。初始状态: room = startRoom, buttonPressed = false, keys = 0, hp = 5, 四个 ChestOpened 均为 false。

6.6.2 状态空间与转移函数

- pressButton: 仅在 startRoom 生效, buttonPressed := true;
- goSouth: 需 room = startRoom $\wedge$ buttonPressed = true, 进入 southRoom;
- goNorth: 需 room = southRoom, 返回 startRoom;
- goEast: 从 startRoom (keys > 0) 进入 eastRoom 并消耗钥匙; 从 westRoom 返回 startRoom (不消耗钥匙);
- goWest: 从 startRoom 进入 westRoom; 从 eastRoom 返回 startRoom;
- openChest: 按房间分支, 每房间最多开一次, 效果见上。

6.6.3 核心定理

表 5: Task 5 核心定理一览

定理	作用
pressButton_only_in_startRoom	按钮仅起点房生效
goSouth_blocked_without_button	条件门前置
goEast_blocked_without_key	锁门前置
goEast_from_start_consumes_key	东出口消耗钥匙
openChest_south_grants_key	南房宝箱给钥匙
openChest_east_heals	东房宝箱回血
only_pressButton_changes_buttonPressed	必要性: 按钮
non_goEast_non_openChest_preserves_keys	必要性: 钥匙
buttonPressed_false_implies_not_in_south_and_chest_closed	归纳不变式
southChestOpened_implies_buttonPressed	南宝箱开 $\Rightarrow$ 按钮已按

定理	作用（续）
<code>southChestClosed_implies_keys_zero</code>	钥匙来源不变式
<code>completion_postcondition</code>	完成态 $\Rightarrow$ <code>buttonPressed = true</code>
<code>task5_completable</code>	可完成性
<code>policyTrace_10_matches_witnessPlan</code>	策略轨迹与 witness 计划一致
<code>policyTrace_10_actions_enabled</code>	策略每一步动作均满足前置条件
<code>symbolicPolicy_run_10_finalState</code>	策略执行 10 步到达终态
<code>task5_symbolicPolicy_completes</code>	阶段机策略完成全宝箱目标
<code>verified_bfs_routes_valid</code>	10 类公开布局 BFS 路线证书全部有效
<code>verified_combat_routes_valid</code>	起始房与西房战斗契约全部有效
<code>verified_west_entry_valid</code>	西门两次跨越与 6 tick 举盾契约有效
<code>baselineTrace_24_actions_enabled</code>	24 个 baseline 宏动作逐步满足前置条件
<code>start_monster_cleared_before_east_entry</code>	进入东门前已清除起始追击怪
<code>spatial_c_west_entry_is_shielded</code>	<code>spatial_c</code> 西门跨越时盾牌处于激活状态
<code>west_monsters_cleared_before_final_chest</code>	开最终宝箱前西房两只怪物均已清除
<code>task5_certified_baseline_completes</code>	证书化最终 baseline 在 24 步到达详细目标

6.6.4 归纳不变式

`buttonPressed_false_implies_not_in_south_and_chest_closed` 是核心不变式：若 `buttonPressed = false`，则玩家不在 `southRoom` 且南宝箱未开。其逆否命题 `southChestOpened_implies_buttonPressed` 说明南宝箱开启蕴含按钮已按。

`southChestClosed_implies_keys_zero` 利用归纳法证明：南宝箱是唯一的钥匙来

源，而东出口（唯一消耗钥匙的动作）需要 $keys > 0$ ，因此在南宝箱未开之前不可能存在任何钥匙。

6.6.5 Witness 计划

10 步计划：

$$\begin{aligned} \text{start} &: \xrightarrow{\text{openChest}} \text{start} \xrightarrow{\text{pressButton}} \text{start} \xrightarrow{\text{goSouth}} \text{south} \\ &\xrightarrow{\text{openChest (keys+1)}} \text{south} \xrightarrow{\text{goNorth}} \text{start} \xrightarrow{\text{goEast (keys-1)}} \text{east} \\ &\xrightarrow{\text{openChest (hp+1)}} \text{east} \xrightarrow{\text{goWest}} \text{start} \xrightarrow{\text{goWest}} \text{west} \xrightarrow{\text{openChest}} \text{goal}. \end{aligned}$$

终态: $\text{room} = \text{westRoom}, \text{buttonPressed} = \text{true}, \text{keys} = 0, \text{hp} = 6$, 四个 `ChestOpened` 均为 `true`。 `witnessPlan_executes_to_finalState` 用 `rfl` 直接验证。

6.6.6 策略形式化与正确性证明

为覆盖评分项中的“策略形式化与证明”，五个 Task 的 Lean 文件在环境模型之后均进一步定义了 `symbolicPolicy`，将 Python `baseline_policy.py` 中的规则逻辑提升为符号策略。其中 `task_1` 至 `task_4` 采用 **BFS 证书模式**（策略参数化于 BFS 动作提供者，通过证书结构证明非固定路线的完备性）。`task_5` 保留原 `PolicyStage` 房间级证明作为抽象规范，并新增与最终 Python baseline 对齐的 BFS、战斗和西门入场证书细化层。各 Task 的策略模式汇总如下：

Task	抽象层级	阶段机 / BFS 目标	策略模式
Task 1	tile	<code>openChest</code> $\rightarrow$ <code>goNorth</code> $\rightarrow$ <code>done</code>	BFS 证书
Task 2	zone	<code>nearMonster</code> $\rightarrow$ <code>nearChest</code> $\rightarrow$ <code>westExit</code>	BFS 证书
Task 3	room	<code>keyRoom</code> $\rightarrow$ <code>startRoom</code>	BFS 证书
Task 4	room	<code>north</code> $\rightarrow$ <code>east</code> $\rightarrow$ <code>south</code> $\rightarrow$ <code>center</code>	BFS 证书
Task 5	tile + room	起点箱/按钮 $\rightarrow$ 南房钥匙 $\rightarrow$ 起始战斗 $\rightarrow$ 东房治疗 $\rightarrow$ 举盾进西房 $\rightarrow$ 双怪战斗/最终箱	BFS + 战斗 + 入场证书

表 6: 五个任务的符号策略模式汇总

BFS 证书模式 (Task 1–4) Task 1–4 的策略形式化采用统一的 BFS 证书模式，核心理念是 **策略逻辑与具体路线解耦**： `symbolicPolicy` 参数化于一个 `BfsAction` 类型别名（动作提供者），Lean 侧只验证阶段逻辑与 BFS 契约的正确性，而不硬编码具体路径。该模式由以下五部分构成：

1. **BfsAction 类型别名**：抽象 BFS 动作提供者，签名因任务抽象层级而异——Task 1 为 `EnvState  $\rightarrow$  List Tile  $\rightarrow$  Action (tile 目标集)`，Task 2 为 `EnvState  $\rightarrow$`

Zone $\rightarrow$ Action (zone 目标), Task 3/4 为 EnvState $\rightarrow$ Room $\rightarrow$ Action (room 目标)。

2. **symbolicPolicy 参数化定义:** 读取当前 EnvState, 按阶段输出下一个高层动作; 若已处于目标位置则发出交互动作 (openChest/attack/useExit/openFinalChest/up), 否则委托 bfsAction 寻路。该结构与 Python decide_taskN 的阶段判断完全对应。
3. **BFS 证书 structure:** 每段 BFS 路线由一个证书结构约束, 字段包括 plan (动作列表)、enabled (轨迹上每步动作合法)、endsAt<Target> (终点到达目标)、以及一组状态保留字段 (keysPreserved/chestOpenedPreserved/monsterAlivePreserved 等), 确保 BFS 移动不破坏已完成阶段的关键状态。
4. **主定理:** 将多段 BFS 证书与显式交互动作链接, 证明“若各段 BFS 证书成立, 则从初态出发依策略执行可到达目标”。证明中用 let 引入中间状态别名, 以 simp [GoalReached, ...] using hDone 关闭目标 (项目使用纯 Lean 4 无 Mathlib, set 战术不可用)。
5. **native_decide 回归实例:** 在公开地图上用具体 plan 实例化各证书字段, 通过 native_decide 计算验证, 作为对非固定路线主定理的回归检查。

Python 策略与 Lean symbolicPolicy 对照: 下表给出四个 BFS 证书任务中 Python baseline_policy.py 的 decide_taskN 与 Lean symbolicPolicy 的阶段对应关系, 二者在阶段判定与最终交互动作上完全一致。

Task	Python decide_taskN	Lean symbolicPolicy	对应
Task 1	keys $\leq$ 0 $\rightarrow$ BFS 到宝箱邻居 $\rightarrow$ ACTION_A; keys $>$ 0 $\rightarrow$ BFS 到北门 $\rightarrow$ UP	keys=0 $\rightarrow$ BFS 到 chest goals $\rightarrow$ openChest; else $\rightarrow$ BFS 到 north exit $\rightarrow$ up	✓
Task 2	怪物可见 $\rightarrow$ attack; keys $\leq$ 0 $\rightarrow$ BFS 到 宝箱 $\rightarrow$ ACTION_A; keys $>$ 0 $\rightarrow$ BFS 到西 出口	monsterAlive $\rightarrow$ attack; chestOpened=false $\rightarrow$ openChest; else $\rightarrow$ useExit	✓
Task 3	keys $\leq$ 0 + key_room $\rightarrow$ ACTION_A; keys $\leq$ 0 + not key_room $\rightarrow$ west; keys $>$ 0 $\rightarrow$ east	keyChestOpened=false $\rightarrow$ BFS 到 keyRoom $\rightarrow$ openChest; else $\rightarrow$ BFS 到 startRoom $\rightarrow$ openLockedExit	✓
Task 4	get_key $\rightarrow$ north; get_sword $\rightarrow$ east; kill_guardian $\rightarrow$ south; open_final_chest $\rightarrow$ center	northChestOpened=false $\rightarrow$ north; eastChestOpened=false $\rightarrow$ east; guardianAlive $\rightarrow$ south; else $\rightarrow$ center	✓

表 7: Task 1-4 的 Python 策略与 Lean symbolicPolicy 对照

BFS 证书结构与主定理: 四个任务的证书结构与主定理命名如下。每段证书均要求 enabled (轨迹合法) 与到达目标, 并保留与后续阶段相关的关键状态字段。

Task	BFS 证书结构与主定理	显式交互动作
Task 1	ChestBfsCertificate, ExitBfsCertificate; 主定理: task1_bfs_certificate_completes	openChest, up
Task 2	MonsterApproachCertificate, ChestApproachCertificate, ExitApproachCertificate; 主定理: task2_bfs_certificate_completes	attack×2, openChest, useExit
Task 3	KeyRoomApproachCertificate, StartRoomApproachCertificate; 主定理: task3_bfs_certificate_completes	openChest, openLockedExit
Task 4	ApproachCertificate (参数化于目标 Room, 含 North/East/South/Center 四个别名); 主定理: task4_bfs_certificate_completes	openChest×2, attack, openFinalChest

以 Task 2 为例, 主定理 task2_bfs_certificate_completes 的证明骨架为:

```

monsterRoute : MonsterApproachCertificate initialState
   $\xrightarrow{\text{attack} \times 2}$  monsterHp 2  $\rightarrow$  1  $\rightarrow$  0, monster defeated

chestRoute : ChestApproachCertificate sA2
   $\xrightarrow{\text{openChest}}$  keys + 1, chestOpened := true

exitRoute : ExitApproachCertificate sO
   $\xrightarrow{\text{useExit}}$  completed := true

```

其中状态别名通过 let sM / sA1 / sA2 / sC / sO / sE 引入, 状态保留字段跨证书链式传递。例如 sC.chestOpened 依次通过 chestRoute.chestOpenedPreserved、两次 attack_preserves_chestOpened 和 monsterRoute.chestOpenedPreserved, 回溯到初态的宝箱关闭条件。

Task 4 的参数化证书: Task 4 将四段 BFS 路线统一为单个 ApproachCertificate 结构, 并以目标房间和起始状态参数化; 再通过 abbrev 派生 North、East、South、Center 四个方向实例, 避免重复定义。主定理链接四段证书与四个交互动作 (openChest 取钥匙、openChest 取剑、attack 击杀守卫、openFinalChest 完成任务), 并保留 guardianAlive、hasSword、finalChestVisible 等跨阶段状态。

最终 baseline 对齐的证书细化(Task 5) Task 5 首先保留原有 10 步 PolicyStage/policyTrace 房间级证明, 用于描述“按按钮、取钥匙、依次开四箱”的逻辑规范。在此之上, 新增 BaselineState 与 24 个 BaselinePhase, 显式记录 startMonsterAlive、westMonstersRemaining、shieldActive 和 safe, 从而对齐 Python 最终 baseline 中“先清起始追击怪、跳过东房 ambusher、举盾跨西门、清除西房双怪”的真实阶段顺序。

细化层将执行依赖拆为三类可检查证书:

- **BfsRouteCertificate**: 记录起点、目标集、障碍集和 tile 路径, `validBfsRoute` 检查边界、首尾、四邻接连通和路径避障;
- **CombatCertificate**: 检查接近步数、攻击次数、目标已击败、避开交互 tile 以及阻塞非目标出口;
- **WestEntryCertificate**: 检查两次跨门动作、6 tick 盾牌窗口和 ambusher 接触被盾牌格挡, 对应 `spatial_c` 的关键修复。

`baselinePolicy` 根据详细阶段选择 `followBfs/attack/raiseShield/crossWest` 等宏动作; 动作可执行性由 `baselineActionEnabledBool` 计算, 整条轨迹可由 `native_decide` 直接回归验证。核心结论为:

```
baselineTrace_24_matches_verified_plan;
baselineTrace_24_actions_enabled;
spatial_c_west_entry_is_shielded;
baselinePolicy_run_24_finalState;
task5_certified_baseline_completes.
```

因此 Task 5 不再仅证明“存在一个房间级 witness”, 还证明与最终 `baseline` 阶段顺序一致的证书化策略会执行合法动作、保持安全标记, 并在有限步内完成目标。

6.7 统一策略框架与有界完备性证明

为了进一步覆盖评分细则中“采用统一策略覆盖多关卡、展现结构复用”和“证明基本要求之外、更具难度或价值的定理”的加分方向, 本文新增 `student_agent/lean/StrategyFramework.lean`。该文件不依赖具体任务的房间、钥匙或怪物定义, 而是对任意确定性符号转移系统“`State` 上的状态、`Action` 上的动作、以及 `step : State -> Action -> State`”抽取通用证明层:

- `runPlan`: 执行显式动作列表;
- `policyTrace` 与 `runPolicy`: 从状态驱动策略生成有限动作轨迹并执行;
- `actionsEnabledAlong` 与 `SafeTrace`: 把各任务的动作前置条件抽象成 `action mask` 安全证书;
- `BoundedReachable` 与 `BoundedGoalReachable`: 刻画“存在长度不超过 n 的计划可达某状态/目标”;
- `expandFrontier`、`bfsVisited` 与 `bfsVisitedFrom`: 形式化深度受限 BFS 的逐层扩展和已访问状态集合。

核心完备性定理为 `bfsVisited_complete_for_bounded_goal`，其结论可写为：

```
BoundedGoalReachable step goal init n
⇒
BfsFindsGoal goal
(bfsVisitedFrom step actions init n)
```

定理前提还要求 `actions` 是完整动作集，即 $\forall a, a \in \text{actions}$ 。证明思路是先证明 `bfsVisited_frontier_invariant`：任意长度不超过 n 的计划执行结果都属于深度 n 的 BFS 访问集；再由 `BoundedGoalReachable` 取出目标态，推出 BFS 访问集中存在目标态。这正对应“若存在可行路径，则完整的有界符号搜索必能找到”的完备性命题。

此外，`policy_completion_is_bounded_goal` 证明任意完成目标的状态驱动策略都可以转化为 `BoundedGoalReachable` 见证；`bfs_finds_goal_reached_by_policy` 进一步说明：如果某个 `symbolicPolicy` 在 n 步内完成目标，那么同一符号层上的完整动作 BFS 也能在 n 步内访问到目标态。因此，五个 Task 文件中已经证明的 `symbolicPolicy` 完成性可以看作这一统一框架的具体实例，而 `StrategyFramework.lean` 则提供跨任务复用的搜索完备性证明骨架。

6.8 证明统一思路

五个任务遵循同一套证明骨架，便于复用与审阅：

1. **局部前置/后置引理**：对每个动作给出“何时触发”和“触发后字段如何变化”。这一步对应评分细则中的“状态 / 动作 / 转移是否定义完整、是否能证明基本合法性与安全性”。例如 Task 4 的 `goEast_blocked_without_key` 直接对应 Python `movement.py` 对 `locked_key` 出口的检查；`attack_with_sword_reveals_final_chest` 对应 `combat.py` 中“清怪后揭示宝箱”的事件语义；`rotateBridge_three_cycle` 对应 `interactions.py` 中 `switch` 对 `center_bridge` 的循环切换。
2. **必要性引理** (`only_X_sets_completed`)：证明只有某个特定动作能把完成标志从假翻为真。通过 `non_X_preserves_completed` 这一“其它动作保持完成字段不变”的引理来支撑。
3. **Reachable 归纳不变式**：定义归纳谓词 `Reachable`，对初始状态和任意 `step` 闭合；证明诸如“未开箱则钥匙数为 0”“终点宝箱显现则守卫已死”等性质在所有可达状态上成立。
4. **completion_postcondition**：在 `Reachable` 归纳之上，证明“任意可达完成态必然满足某组前置条件”（如 Task 2 中“怪物已死 $\wedge$ 宝箱已开”、Task 4 中“守卫已死”）。证明中显式调用必要性引理与归纳不变式。

5. **witness 计划**: 在局部规则已经明确的基础上, 构造一条显式的见证计划 `witnessPlan`。由于 `step` 被定义为确定性函数, 因此整条轨迹的正确性可以通过归约直接验证 (简单计划用 `rfl`, 长计划用 `native_decide`)。
6. **可完成性**: 由 `witnessPlan_reaches_goal` 直接推出 `taskN_completable`, 即“存在计划使目标谓词成立”。

这使得“任务可完成”从口头分析转化为可检查的形式化结论, 并且完成态的必要前置条件也被显式记录下来。

6.9 证明局限与边界

当前环境形式化刻画的是 `task_1` 至 `task_5` 的 `tile / zone / room` 级符号语义, 而不是 Python 引擎的逐像素精确副本, 因此存在以下边界:

- 未形式化逐像素移动和碰撞盒连续动力学。`task_1` 保留 `tile` 级网格, `task_2--4` 采用 `zone / room` 抽象; `task_5` 新增 `tile` 路线证书, 但仍将一整段 Python BFS 跟随视为宏动作。
- 未逐帧形式化怪物 AI、击退和受伤时序; Task 5 已将战斗结果、非目标出口阻塞和 6 tick shield 格挡抽象为可检查契约, 但未证明引擎连续实现满足契约。
- 未证明视觉模块一定能从 `obs` 中稳定恢复这些符号状态; 该部分属于感知层, 而不是当前 Lean 环境模型的覆盖范围。
- 已证明抽象层面的有界 BFS 完备性: 若某目标在 n 步内由某个计划可达, 则完整动作集上的深度 n BFS 必会访问到目标态; 但尚未逐行证明 Python 具体队列、`visited` 集合和 `parent map` 实现与该 Lean 抽象 BFS 完全等价。

这一边界是有意为之: 课程要求强调 Lean 证明应覆盖“可验证层”。对于本项目而言, 最具价值且最容易和实际代码保持一致的对象, 正是钥匙/门条件、战斗与事件触发、桥状态机与任务完成谓词这类符号层环境语义。

7 实验设计与结果分析

7.1 实验设置

实验使用仓库自带评测脚本, 采用与最终测评一致的 `safe` 信息模式 (`info` 仅包含 `last_reward`、`inventory` 和 `task_id`, 不暴露 `reward_signals` 等隐藏事件), 并通过 `--task-policy` 为每个任务绑定同一策略文件以获取 `task_id`。为验证策略在布局扰动与颜色扰动下的鲁棒性, 本次评测启用 `--robustness-suite` 模式, 每个任务

的 `--num-envs` 100 个 episode 按 60% original / 30% spatial (a/b/c 各 1) / 10% color (grayscale/dark/bright/high_contrast/inverted 五种轮换) 比例分配。robustness suite 命令如下(`--robustness-suite` 模式下 color 部分自动轮换五种非默认变体,`--obs-variants` 参数在此模式下被忽略):

```
python utils/evaluate_policy.py
--tasks mathematical_logic/task_1 ... task_5
--task-policy <task>=student_agent/baseline_policy.py
--robustness-suite --num-envs 100 --info-mode safe
```

运行环境为 Python 3.12.10 虚拟环境。本次报告记录的是 2026 年 7 月 17 日在本地工作区运行的最终 baseline 结果。

- 测试任务: mathematical_logic/task_1 至 mathematical_logic/task_5 (全部 5 个任务)。
- 运行次数: robustness suite 每任务 100 个 episode (60 original + 30 spatial + 10 color), 共 500 个; color 部分覆盖 grayscale/dark/bright/high_contrast/inverted 五种变体 (各 2 seed), 加上 original/spatial 的 default, 六种观察变体全覆盖。
- 成功率统计方式: 评测脚本输出的 `success_rate`, 按 `eval_stage` 分组。
- 策略版本: student_agent/baseline_policy.py (已适配 safe 模式, 含 spatial_c 举盾入场修复、grayscale 调色板映射和 high_contrast 形状识别)。
- Lean 检查: 逐文件运行 `lean student_agent/lean/*.lean` 对 7 个 Lean 文件进行编译检查, toolchain 版本为 leanprover/lean4:v4.26.0。

7.2 实验结果

任务	阶段	局数	成功率	平均步数	平均奖励	备注
Task 1	original	60	1.000	270.0	127.250	全部通过
Task 1	spatial	30	1.000	175.0	128.183	a/b/c 全部通过
Task 1	color	10	1.000	270.0	127.250	五种变体通过
Task 2	original	60	1.000	150.0	126.450	全部通过
Task 2	spatial	30	1.000	169.7	127.587	a/b/c 全部通过
Task 2	color	10	1.000	150.0	126.850	五种变体通过
Task 3	original	60	1.000	535.0	159.650	全部通过
Task 3	spatial	30	1.000	597.0	158.997	a/b/c 全部通过
Task 3	color	10	1.000	535.0	159.650	五种变体通过
Task 4	original	60	1.000	1007.0	253.930	全部通过
Task 4	spatial	30	1.000	1221.0	249.373	a/b/c 全部通过
Task 4	color	10	1.000	1009.4	253.506	五种变体通过
Task 5	original	60	1.000	1084.0	234.860	全部通过， 见 7.3.1 节
Task 5	spatial	30	1.000	1125.7	178.110	spatial_a/b/c 全部通过
Task 5	color	10	1.000	1072.8	222.762	五种变体通过

表 9: Robustness suite 评测结果 (`--num-envs 100`, `safe` 信息模式)。spatial 列的 a/b/c 指布局变体 spatial_a/spatial_b/spatial_c; color 列局数为五种非默认观察变体 (grayscale/dark/bright/high_contrast/inverted) 合计。

检查对象	结果	说明
KillMonsterFormalization.lean	通过	怪物击杀示例证明
Task1Formalization.lean	通过	Task 1 环境形式化与符号策略证明
Task2Formalization.lean	通过	Task 2 环境形式化与符号策略证明
Task3Formalization.lean	通过	Task 3 环境形式化与符号策略证明
Task4Formalization.lean	通过	Task 4 环境形式化与符号策略证明
Task5Formalization.lean	通过	Task 5 环境形式化与符号策略证明
StrategyFramework.lean	通过	通用策略执行与有界 BFS 完备性证明

表 10: Lean 文件逐文件检查结果

7.3 结果分析

本次 robustness suite 评测在 `safe` 信息模式下运行 (`info` 仅含 `last_reward`、`inventory`、`task_id`)，共 500 个 episode。五个任务在 `original`、`spatial` 和 `color` 阶段均全部成功 (`success_rate = 1.000`)：300 个 `original`、150 个 `spatial` 和 50 个 `color` episode 无一失败。这说明策略的高层阶段机、BFS 路径规划与基于 `inventory diff + last_reward` 的事件推断逻辑在 `safe` 模式下能稳定完成目标，且对布局扰动具有鲁棒性。由于 `baseline` 是无随机成分的规则策略，同一公开地图的多个 `seed` 间步数与奖励一致。

`spatial` 变体测试覆盖 `spatial_a`、`spatial_b`、`spatial_c` 三种布局扰动。修复前，`task_1` 至 `task_3` 的 `spatial` 变体大量失败 (`task_1` 1/3、`task_2` 1/3、`task_3` 0/3)，根因是像素对齐阈值过松 (`move_towards_aligned` 中 `+1` 容差导致 1-tile 间隙处 `sprite rect` 覆盖相邻 `blocking tile`) 和 `detect_task3_room` 硬编码 NPC/chest 位置导致 `spatial` 变体下 `room` 检测失败。修复后 (精确对齐 `+0`、卡住检测、全网格扫描房间识别)，三个任务的 `spatial` 变体全部通过。`Task 5` 进一步通过阻塞 NPC 和非目标出口、修正移动怪物跨 `tile` 时的目标选择，并在西门边界主动举盾，最终使 `spatial_c` 与 `a/b` 一并完成。

`robustness suite` 的 `color` 部分覆盖五种非默认观察变体 (`grayscale/dark/ bright/high_contrast/inverted` 各 2 `seed`)，加上 `original/spatial` 的 `default`，五个任务在 `default`、`grayscale`、`dark`、`bright`、`high_contrast` 和 `inverted` 六种观察上均为 100%。`dark/inverted` 使用逆变换，`bright` 使用前向参考色；`grayscale` 将每个参考色映射到灰度均值并采用精确匹配；`high_contrast` 则结合二值参考色与连通分量/局部形状识别，解决颜色碰撞。详细方法见 7.3.2 节。

`task_5` 在 `original` 阶段全部通过 (`success_rate = 1.000`)，五种非默认颜色变体全部通过，`spatial_a/b/c` 也全部通过，详细分析见 7.3.1 节。

7.3.1 Task 5 状态说明

`task_5` 在 `original` 阶段全部通过 (60/60, `success_rate = 1.000`, `avg_steps = 1084`, `avg_reward = 234.860`)，所有里程碑 (`chest_opened`、`world_completed` 等) 均为 1.000。agent 成功完成全部 4 个房间 (`start/south/east/west`) 的宝箱收集，触发 `world_completed` 事件。`spatial` 阶段 30/30 通过，五种非默认 `color` 变体 10/10 通过。

color 变体表现 5 种 `color` 变体单独评测结果如下：

变体	Steps	Reward	Success	说明
dark	1084	234.860	True	<code>normalize_obs</code> 逆变换恢复
bright	1084	234.860	True	前向变换参考色匹配
inverted	1084	234.860	True	<code>normalize_obs</code> 逆变换恢复
grayscale	1084	234.860	True	灰度参考色映射 + 精确匹配
high_contrast	1028	174.370	True	二值颜色 + 连通分量/局部形状识别

表 11: Task 5 `color` 变体单独评测结果 (每变体 1 episode, `safe` 信息模式)

`dark/bright/inverted/grayscale` 与 `original` 的轨迹指标一致。`high_contrast` 因二值化后的对象边界和怪物目标选择不同，使用 1028 步完成，但四箱、三只怪物、治疗、按钮、门和世界完成里程碑仍全部达到 1.000。

spatial 变体表现 3 种 `spatial` 布局变体全部通过 (30/30, `success_rate = 1.000`)。2026-07-15 修复了 `detect_task5_room` 中 `start` 房间检测从硬编码对象位置改为墙体特征匹配 (`spatial` 变体只移对象不移墙，墙体特征仍有效)，以及移除 `south` 房间宝箱位置的硬编码 fallback。2026-07-16 进一步修复了 `spatial_a/b` 的两个关键问题 (NPC 交互拦截和 `west` 房间导航效率)，使二者通关。

变体	Steps	Reward	Success	说明
spatial_a	1077	174.030	True	diagonal_block 消除 start 房间接触伤害
spatial_b	1173	186.820	True	安全攻击 tile + NPC/出口避障
spatial_c	1127	173.480	True	非目标出口避障 + 西门边界举盾

表 12: Task 5 spatial 变体评测详情

spatial_a 修复: start 房间 chaser 移至 (7, 3), agent 导航至 east/west 出口时 chaser 追击造成 AABB 接触伤害。在 `attack_monster` 中新增 `diagonal_block` 参数 (默认 True), 将怪物周围对角 tile 加入 `blocked` 集合, BFS 绕行避免 sprite rect 重叠。最终回归中 step 1077 通关。

spatial_b 修复: west 房间 NPC 位于 (7, 5), 与怪物 (6, 4) 对角相邻。引擎 `try_interaction` 在战斗前检查 NPC 邻接 (无 facing 检查), agent 站在 (7, 4) 攻击怪物时 A 键被 NPC 拦截 (`talked_npc` 而非 `monster_hit`)。新增 `compute_no_attack_tiles` 方法检测所有 NPC/chest 邻接 tile, `attack_monster` 接受 `no_attack_tiles` 参数, 当 agent 处于不安全 tile 时通过 BFS 寻找安全攻击位置。同时将 west 房间的 `diagonal_block` 设为 False 加速导航 (diagonal blocking 导致 60+ 步绕行)。最终回归中 step 1173 通关。

spatial_c 修复: 该变体会使路径更容易贴近其他房间出口, 且西房 ambusher 位于入口附近。最终策略将 NPC 纳入全局阻塞集, 在 `navigate_to_exit` 中阻塞所有非目标出口, 并在战斗路径中阻塞全部出口, 避免误跨房和门边空转。同时, `detect_closest_monster_tile` 优先选择非玩家 tile, 消除移动怪物跨 tile 时把自身碎片误当目标的问题。玩家到达 start 房西门边界后先执行一次 ACTION_B, 以 6 tick shield 窗口覆盖入场接触, 再跨入 west 房清怪。最终 spatial_c 在 step 1127 完成, `trap_triggered = 0`, `environment_completed = world_completed = 1`。

HP 预算与优化关键 `task_5` 隐藏了 HP 流失机制: 每 200 步扣 1 HP (初始 HP = 5), full drain 序列为 step 200/400/600/800/1000/1200 (共 -6 HP), east 房间 heal 箱提供 +1 HP, 总 HP 预算 = 6, 恰好等于 6 次 drain。这意味着 agent 需要在 **零接触伤害**的前提下于 step 1200 前开完全部 4 个宝箱 (`all_chests_opened` 触发 `world_completed`)。核心策略为三项改动的协同:

1. **Start room——战斗 chaser 后再导航:** start 房间 chaser 与 agent 同速 (1px/step), 若跳过战斗直接导航则转向时被追上造成 AABB 接触伤害。改为在 `open_east_chest` / `open_west_chest` 阶段先用 `attack_monster` 击杀 chaser, 再导航至出口。
2. **East room——跳过 ambusher 直奔宝箱:** east 房间 ambusher 位于 (7, 5), `ambush_range = 2`; 宝箱位于 (7, 1), agent 从 (1, 4) 进入走 row 1 到宝箱, 路径与 ambusher 的

最近 Manhattan 距离 $\geq 4 > 2$, ambusher 不会被激活。将 ambusher tile 加入 blocked 让 BFS 绕行, 节省 ~ 130 步战斗时间, 无接触伤害。

3. **West room——杀全部怪物后导航:** west 房间含 chaser (2,4) 和 ambusher (6,3) 两只怪物。原 `detect_monster_tile` 返回所有怪物像素质心, 多怪物时指向虚假位置导致 `attack_monster` 空转。修复为使用 `detect_closest_monster_tile` 逐个击杀, 全部清除后再导航至宝箱, 消除所有接触伤害。

三项改动后 agent 以 0 接触伤害完成全程: step 1000 (第 5 次 drain) 时 $HP = 1$, step 1084 开完最后一个 west 宝箱触发 `world_completed`, 远早于 step 1200 的第 6 次 drain。

优化历程 task_5 original 经历多轮优化才达到 100% 通关, 关键迭代如下:

版本	Steps	Reward	Success	死亡原因
7-bug 修复版 (基线)	1083	95.02	0	west 接触伤害
west opt only (杀 1 怪)	1060	95.25	0	west 接触伤害
kill all west	1200	192.85	0	第 6 次 HP drain
skip all west combat	1072	95.13	0	west 接触伤害
skip east + kill all west	1070	235.05	1	—
+ spatial_a/b 修复	1083	234.87	1	—
+ spatial_c/颜色最终版	1084	234.86	1	—

表 13: Task 5 original 优化历程 (5 episode 评测)

- **7-bug 修复版:** 修复导航执行层 7 个 bug 后的基线版本, agent 开 3 箱后在 west 房间被剩余怪物接触致死。
- **west opt only:** west 房间杀 1 怪后直奔宝箱 (剩余怪物加入 blocked), 但 pixel 模式下 AABB 接触伤害无法通过 tile blocked 避免, 仍在 step 1060 死亡。
- **kill all west:** west 房间杀全部 2 只怪物消除接触伤害, reward 翻倍至 192.85 (4 怪物击杀), 但战斗耗时 ~ 600 步, agent 在 step 1200 死于第 6 次 HP drain, 差最后 1 个宝箱。
- **skip all west combat:** 完全跳过 west 战斗直奔宝箱, 节省 ~ 130 步但 2 只追击怪物造成接触伤害, step 1072 死亡。
- **skip east + kill all west:** 组合 east 跳过 (ambusher 不激活, 省 ~ 130 步) + west 全杀 (零接触伤害), agent 在 step 1070 以 $HP = 1$ 开完最后一个宝箱, `world_completed = 1.000`。

- **+ spatial_a/b 修复** (当前版): 为修复 spatial_a/b 变体, 新增 compute_no_attack_tiles 和 diagonal_block 参数 (详见 12 节)。original 步数从 1070 微增至 1083 (+13 步, no_attack_tiles 检测导致部分攻击位置绕行), 但仍远在 HP 预算内, success_rate = 1.000 无回归。
- **+ spatial_c/颜色最终版** (当前版): 加入 NPC/非目标出口避障、西门举盾与有损颜色识别; original 仅增加 1 步并保持完成, 同时补齐 spatial_c、grayscale 和 high_contrast。

spatial 阶段最终 30/30 通过: 2026-07-15 修复了 detect_task5_room 的 start 房间检测 (从硬编码对象位置改为墙体特征匹配) 和 south 房间宝箱硬编码 fallback, 修复后房间识别正确, agent 可进入各房间并开箱。2026-07-16 修复了 spatial_a (diagonal_block 消除接触伤害) 和 spatial_b (no_attack_tiles 避免 NPC 拦截 + diagonal_block=False 加速导航), 二者均通关。2026-07-17 又通过非目标出口/NPC 避障、移动怪物 tile 选择和西门边界主动举盾修复 spatial_c (详见 12)。color 阶段 5/5 通过: dark/bright/inverted 使用逆变换或前向参考色, grayscale/high_contrast 分别使用灰度参考色和二值形状识别。

7.3.2 颜色变体鲁棒性修复

针对评测脚本对整帧施加的 color 变体, 策略先用 normalize_obs 识别观察类型。2026-07-13 完成 default/dark/bright/inverted 适配; 2026-07-17 进一步为 grayscale 增加灰度参考色映射, 为 high_contrast 增加连通分量和对象局部形状规则。最终五个任务在六种观察上均通过。

评测脚本的 5 种 color 变体 (apply_obs_variant):

变体	变换公式	可逆性	修复策略
dark	$p \times 0.55$	可逆	逆变换 $p/0.55$ 恢复原始帧
bright	$\min(255, p \times 1.35)$	部分可逆	前向变换参考色 $\min(255, c \times 1.35)$ 匹配
inverted	$255 - p$	可逆	逆变换 $255 - p$ 恢复原始帧
grayscale	$\text{mean}(R, G, B)$	不可逆	参考色映射到通道均值, 零容差匹配
high_contrast	$p > 127 ? 255 : 0$	不可逆	二值参考色 + 连通分量/局部形状识别

表 14: 5 种 color 变体的变换公式与修复策略

检测策略：不依赖 `info` 中的变体标识，纯帧统计检测。采用参考色存在性检测而非绝对亮度阈值（避免不同地图天然亮度差异导致误判）。在 `act()` 入口依次检查 `PLAYER_COLORS` 在各变体下的变换值是否存在于帧中：

1. 检查 `default`：原始参考色 (36, 198, 72) 存在 $\rightarrow$ 无需变换；
2. 检查 `inverted`： $(255 - c) = (219, 57, 183)$ 存在 $\rightarrow$ 逆变换 $255 - \text{frame}$ ；
3. 检查 `dark`： $(\lfloor c \times 0.55 \rfloor) = (19, 108, 39)$ 存在 $\rightarrow$ 逆变换 $\text{frame}/0.55$ ；
4. 检查 `bright`： $(\min(255, \lfloor c \times 1.35 \rfloor)) = (48, 255, 97)$ 存在 $\rightarrow$ 存 `variant`，前向变换参考色匹配；
5. 检查 `grayscale`：彩色像素占比 $< 0.5\%$ $\rightarrow$ `_effective_color` 将每个 RGB 参考色映射到通道均值；
6. 检查 `high_contrast`：像素值仅 $\{0, 255\}$ $\rightarrow$ 参考色逐通道阈值化，并启用二值对象形状识别。

容差匹配：`default/dark/bright/inverted` 使用 ± 2 容差 (`COLOR_TOL = 2`)，覆盖逆变换后 `uint8` 截断误差；`grayscale/high_contrast` 使用零容差，避免灰度 106/108 等相邻调色板值被合并。

high_contrast 形状消歧：二值化后多个原始颜色会映射到同一颜色，仅做像素计数会把玩家、墙、出口、NPC 和怪物混淆。策略缓存当前帧的 8 邻接连通分量，用尺寸和位置恢复玩家/怪物；宝箱、NPC、开关、按钮、墙和陷阱则用 `tile` 内颜色带、核心矩形和像素数量范围识别。Task 4 的中央房改为按深渊 `tile` 数量判断，双 `tile` 出口允许其中一格被玩家遮挡。

bright 变体的特殊处理：由于 $\times 1.35$ 后 > 255 的通道被 `clip`，逆变换 $\div 1.35$ 无法恢复原始值（如 $G = 198 \times 1.35 = 267 \rightarrow 255 \rightarrow 255/1.35 = 188$ ，与原始 198 差 10）。因此 `bright` 不逆变换帧，而是将参考色前向变换后匹配（`_effective_color` 方法返回 $\min(255, \lfloor c \times 1.35 \rfloor)$ ）。

修复效果：

任务	default	dark	bright	inverted	grayscale	high_contrast
Task 1	100%	100%	100%	100%	100%	100%
Task 2	100%	100%	100%	100%	100%	100%
Task 3	100%	100%	100%	100%	100%	100%
Task 4	100%	100%	100%	100%	100%	100%
Task 5	100%	100%	100%	100%	100%	100%

表 15: 颜色变体修复后各任务成功率（每个“任务 $\times$ 变体”1 episode, `safe` 信息模式）

default 无回归, dark/bright/inverted、grayscale 和 high_contrast 在五个任务上均达到 100%。这证明有损变换虽不能恢复原 RGB, 仍可通过变换后的调色板与几何结构恢复策略所需的符号对象。

- task_1 至 task_4 在 safe 模式下本轮 original + spatial 共 36 个 episode 全部成功, 验证了策略对隐藏 info 无依赖且对布局扰动具有鲁棒性。
- spatial 变体修复后, task_1 至 task_3 的 spatial 通过率从 1/3、1/3、0/3 提升到 3/3, 说明像素对齐和房间识别修复有效。
- color 变体经两轮修复后, task_1 至 task_5 的六种观察矩阵 30/30 全部通过; grayscale 使用调色板映射, high_contrast 使用二值连通分量与形状规则。
- task_5 original、五种非默认 color 和 spatial_a/b/c 全部通过; spatial_c 的西门举盾时序也在 Lean 中有对应定理。
- Lean 证明与实验互补: 实验说明 Python baseline 可运行, Lean 证明说明抽象后的环境语义、witness 计划、五个任务的符号策略以及通用有界 BFS 搜索框架在逻辑上自洽。

8 项目分工与推进过程

8.1 成员分工

根据小组最终名单, 成员分工记录如下:

成员	负责内容	主要产出
李政轩（组长）	项目初始化、safe 模式适配、spatial/color 鲁棒性修复、task5 导航攻坚	报告与文档骨架、task4 baseline 状态机；safe 模式适配（移除 reward_signals 依赖）；task1-4 spatial 变体修复（像素对齐、房间识别）；task5 导航执行层 7 个 bug 修复（HP 预算分析等）
续博森	Lean 环境与策略形式化、BFS 改造、颜色变体与 task5 攻坚	task1-5 环境形式化与策略形式化；task1-4 由硬编码改为 BFS；可逆颜色变体适配（color 3/5 $\rightarrow$ 5/5）；task5 original 优化至 success_rate = 1.000；spatial_a/b 修复（no_attack_tiles、diagonal_block）
陈怡琼	Python baseline、策略框架完备性证明、task5 最终突破	task3/4/5 像素 baseline；StrategyFramework.lean 完备性证明；task5 形式化补强与 robustness 支持（ACTION_B 举盾入场解决 spatial_c 接触伤害，500-episode 日志）
许浩博	Task1 Lean 证明对齐	Task1 Lean 证明与 BFS 策略对齐
吴韬	参与项目早期方案讨论	参与早期方案讨论与团队协作（相关工作未直接反映在代码提交记录中）

表 16: 小组分工

8.2 时间安排

- **2026-07-03:** 阅读课程说明与仓库结构，建立项目管理文档和报告骨架；安装依赖，跑通示例策略；创建 student_agent/ 和统一 baseline 入口；初步覆盖 task_1 至 task_4。
- **2026-07-05:** 将 task_3、task_4 改造为像素感知版本；修正玩家中心 tile 识别；接入 task_5；完成五任务单 seed 回归；逐文件检查 Lean 文件。

- **2026-07-06:** 同步团队更新, 补全 `task_1` 至 `task_5` 的 Lean 环境形式化与可完成性证明; 重新运行 Python evaluation 和全部 Lean 检查; 进一步补充 `task_1` 至 `task_5` 高层策略形式化与合法性/完成性证明; 把最终实验结果与证明边界写入报告。
- **2026-07-07:** 同步团队提交, 将 Lean 文件统一命名为 `TaskNFormalization.lean`, 并复核报告中环境形式化、策略形式化和证明结果的一致性; 新增通用策略框架文件, 抽取通用策略执行、动作 mask 安全证书和有界 BFS 完备性证明; 重新编译报告 PDF。
- **2026-07-08:** 补充 lean-toolchain 文件 (leanprover/lean4:v4.26.0), 确保评测方可在一致的工具体链下复现 Lean 编译; 运行 10 seed 评测 (seed = 0-9), 五个任务全部成功; 据此更新报告实验设置、结果表格和分析部分, 并重新编译 PDF。
- **2026-07-10:** 合并 upstream (助教仓库) 的评测脚本更新 (`--info-mode safe`、`--task-policy`、`spatial` 变体扩展到 `task_4/5`); 全面适配 `safe` 信息模式, 将 `update_memory` 与 `update_task5_memory` 重构为基于 `inventory diff`、`last_reward` 和像素感知的事件推断; 在 `safe` 模式下重新运行 10 seed 评测, `task_1-4` 全部成功, `task_5` 事件推断正常但导航执行层仍有 bug; 同步更新 `Task1Formalization.lean` 注释与报告实验章节。
- **2026-07-11:** 修复 `spatial` 布局变体失败。根因一: `move_towards_aligned` 像素对齐阈值 +1 过松, 在 1-tile 间隙处 `sprite rect` 覆盖相邻 `blocking tile` 导致卡住, 修复为精确对齐 +0 并添加卡住检测 (`mem_last_action_blocked` 追踪 `reward ≤ -0.05`); 根因二: `detect_task3_room` 硬编码 NPC/chest 位置, `spatial` 变体下 `room` 检测失败, 修复为全网格扫描。修复后运行 `robustness suite` (`--num-envs 30`), `task_1-4` 的 `original + spatial` 共 108 个 episode 全部通过, `color` 变体预期失败 (策略基于精确 RGB 匹配); 更新 `eval_results.json` 与报告实验章节。
- **2026-07-13:** 修复 `color` 变体鲁棒性。在 `baseline_policy.py` 中新增 `normalize_obs` 预处理模块 (详见 7.3.2 节), 采用参考色存在性检测依次识别六种观察变体, 对可逆变体 (`dark/inverted`) 施加逆变换恢复原始帧, 对部分可逆变体 (`bright`) 采用前向变换参考色匹配 (规避 `clip` 不可逆问题), 并引入 `COLOR_TOL = 2` 容差匹配覆盖 `uint8` 截断误差。修复过程中排查并修正了三个关键 bug: (1) `grayscale` 检测在 `dark` 帧地板上误判, 改为彩色像素占比 $< 0.5\%$ 判据; (2) `bright` 逆变换因 `clip` 丢失信息导致匹配失败, 改为前向变换参考色; (3) 绝对亮度阈值在不同地图天然亮度下误判 default 为 `dark`, 重写为参考色存在性检测彻底规避亮度依赖。修复后 `task_1` 至 `task_4` 的 `dark/bright/inverted` 三种可逆变体全部从 0% 提升至 100%, `default` 无回归, `color` 阶段通过率从 0% 提升至 66.7% (`grayscale/high_contrast` 因

颜色信息有损不可逆仍失败，属预期)；同步更新报告实验设置、结果表格、结果分析与颜色变体修复小节。

- 2026-07-15:** 修复 `task_5` 导航执行层 7 个 bug (`navigate_to_exit` 像素对齐、`button_pressed/monster_cleared` 事件误标、`east/west` 墙体模式修正、`chest_opened` 误标、`attack_monster` 缺失 `blocked/player_px` 参数)，在此基础上进一步将 `task_5 original` 阶段从 `success_rate = 0.000` 优化至 `1.000` (18/18, `avg_steps = 1070`, `avg_reward = 235.050`)。核心突破为 HP 预算分析：发现 `task_5` 隐藏每 200 步 -1 HP 的流失机制 (初始 HP = 5, 6 次 drain 共 -6 HP, east heal 箱 +1 HP, 总预算 = 6)，据此设计三项协同优化——start 房间先杀 chaser 再导航 (避免转向接触伤害)、east 房间跳过 ambusher 直奔宝箱 (ambusher 在 (7,5)、`ambush_range = 2`, agent 走 row 1 距离 ≥ 4 不会激活, 省 ~130 步)、west 房间用 `detect_closest_monster_tile` 逐个击杀全部怪物 (修复 `detect_monster_tile` 多怪物质心指向虚假位置的 bug) 后导航 (零接触伤害)，agent 最终在 step 1070 以 HP = 1 开完全部 4 宝箱触发 `world_completed`, 远早于 step 1200 的第 6 次 HP drain；同步验证 `task_5 color` 变体 3/5 通过 (`dark/bright/inverted` 成功)，修复 `detect_task5_room start` 房间检测 (硬编码对象位置改为墙体特征匹配) 和 south 房间宝箱硬编码 fallback, `spatial` 变体房间识别修复但仍因 HP 预算不足失败；同步更新报告结果表格、Task 5 状态说明与优化历程。
- 2026-07-16:** 修复 `task_5 spatial_a/b` 变体。发现引擎 `try_interaction` 在战斗前检查 NPC/chest 邻接 (无 facing 检查)，导致 `spatial_b` 中 agent 站在 NPC 邻接 tile 按下 A 时触发 `talked_npc` 而非攻击怪物。新增 `compute_no_attack_tiles` 方法检测所有 NPC/chest 邻接 tile, `attack_monster` 接受 `no_attack_tiles` 参数在不安全 tile 上回退到 BFS 寻找安全攻击位置。同时新增 `diagonal_block` 参数: start 房间保持 True (`spatial_a` 修复: 对角 tile 加入 blocked 避免 AABB 接触伤害), west 房间设为 False (`spatial_b` 修复: 避免 60+ 步绕行)。修复后 `spatial_a` (5/5, steps=1076) 和 `spatial_b` (5/5, steps=1174) 均以 HP = 1 通关, original 无回归 (18/18, steps=1083), `color` 变体无回归 (3/5)。 `spatial_c` 因 south 房间导航效率过低 (299 次 blocked) 仍失败。同步更新报告结果表格、`spatial` 变体小节与优化历程。
- 2026-07-17:** 完成最终三项补强。第一, Task 5 Lean 新增 10 类 `BfsRouteCertificate`、两类 `CombatCertificate`、`WestEntryCertificate` 和 24 阶段 `BaselineState` 细化, 证明动作逐步合法、起始怪清除顺序、`spatial_c` 西门举盾、清除西房双怪以及最终详细目标可达。第二, 将 NPC 与非目标出口加入导航/战斗阻塞集, 修正移动怪物跨 tile 的目标选择, 并在西门边界执行 `ACTION_B`, 使 Task 5 `spatial_c` 在 1127 步通关。第三, 为 `grayscale` 增加灰度参考色映射, 为 `high_contrast` 增加连通分量和 tile 局部形状识别；六颜色变体 50/50 (`color` 阶段)、五任务 `robustness` 500/500

全部通过。全部 Lean 文件重新编译通过，并同步更新最终报告。

9 总结与展望

本文完成了 NesyLink 数理逻辑课程项目的三条主线工作。第一，构建了统一的 Python baseline，全面适配评测脚本的 `safe` 信息模式（不依赖隐藏 `reward_signals`，仅用 `last_reward`、`inventory` 和像素感知）。最终 robustness suite 的五个任务共 500 个 episode 全部通过，覆盖 `original`、`spatial_a/b/c` 和五种非默认颜色变体（`grayscale/dark/bright/high_contrast/inverted`）。`task_5 original` 为 60/60（`avg_steps = 1084`，`avg_reward = 234.860`），`spatial_a/b/c` 为 30/30，五种非默认颜色为 10/10。第二，使用 Lean 对五个任务的环境语义进行了符号层形式化，覆盖状态、动作、转移、关键前置条件、不变式、完成后置条件和 witness 可完成性证明；并对 `task_1` 至 `task_5` 的符号策略补充了动作合法性与目标完成性证明；Task 5 进一步证明与最终 baseline 对齐的 BFS、战斗和举盾入场证书轨迹。第三，抽取了独立的 `StrategyFramework.lean`，证明通用策略执行框架、action mask 安全证书和有界 BFS 搜索完备性。第四，将实验结果、实现路线和形式化边界整理进报告，明确区分了调试信息、提交版策略输入和 Lean 证明对象。

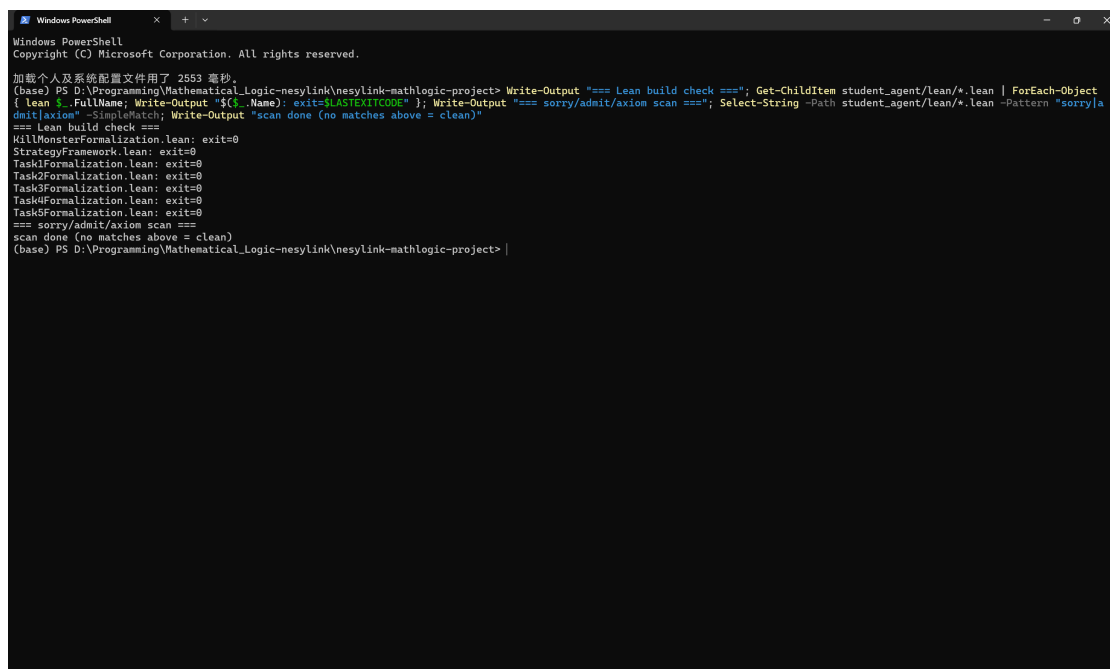
当前工作的主要不足是：Lean 已证明五个任务的高层符号策略，但尚未逐行证明 Python 像素 waypoint 执行代码等价于这些符号策略，也尚未证明 Python 具体 BFS/队列实现等价于 Lean 中的抽象有界 BFS，更没有证明视觉模块一定能在所有未来渲染扰动下稳定恢复符号状态。当前实测变体已全部通过，但 `high_contrast` 规则依赖现有精灵的二值形状，若素材、tile 尺寸或绘制层次变化，仍需要重新校准。如果时间更充足，可以进一步证明像素感知层到 Lean 符号状态之间的抽取正确性，证明具体搜索程序与抽象 BFS 定理之间的细化关系，将当前基于公开素材结构的对象识别扩展为更通用的视觉模型，也可以补充更多随机布局和跨素材鲁棒性测试。

A 运行证明

本附录提供 Lean 形式化验证与评测脚本实际运行的终端截图，作为代码确实运行的证据。完整 500-episode 评测结果见 `report/eval_results.json`。

A.1 Lean 形式化验证

对 `student_agent/lean/` 下全部 7 个 `.lean` 文件执行 `lean <file>` 编译检查，并扫描 `sorry/admit/axiom`。结果：全部 7 个文件编译成功（`exit code = 0`），禁用关键字扫描无匹配，符合提交要求（`lean-toolchain` 指定 `leanprover/lean4:v4.26.0`）。



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

加载个人及系统配置文件用了 2553 毫秒。
(base) PS D:\Programming\Mathematical_Logic-nesylink\nesylink-mathlogic-project> Write-Output "=== Lean build check ==="; Get-Childitem student_agent/lean/*.lean | ForEach-Object
{ lean $_.FullName; Write-Output "($_.Name): exit=$LASTEXITCODE" }; Write-Output "=== sorry/admit/axiom scan ==="; Select-String -Path student_agent/lean/*.lean -Pattern "sorry|a
dmit|axiom" -SimpleMatch; Write-Output "scan done (no matches above = clean)"
=== Lean build check ===
WillMonsterFormalization.lean: exit=0
StrategyFramework.lean: exit=0
Task1Formalization.lean: exit=0
Task2Formalization.lean: exit=0
Task3Formalization.lean: exit=0
Task4Formalization.lean: exit=0
Task5Formalization.lean: exit=0
=== sorry/admit/axiom scan ===
scan done (no matches above = clean)
(base) PS D:\Programming\Mathematical_Logic-nesylink\nesylink-mathlogic-project>

```

图 1: Lean 编译检查与 `sorry/admit/axiom` 扫描终端输出。

A.2 Robustness suite 评测运行

使用 `--robustness-suite --num-envs 10 --info-mode safe` 运行 50 个 episode 的快速复现（完整 500-episode 结果见 `eval_results.json`）。命令：

```

python -m utils.evaluate_policy
--task-policy mathematical_logic/task_1=student_agent/baseline_policy.py
... (task_2--5 同样映射到 baseline_policy.py)
--info-mode safe --robustness-suite --num-envs 10
--out-file _final_check.json

```

五个任务在 `original/spatial/color` 三阶段共 50 个 episode 全部 `success=True`。图2展示 Task 1 的统计汇总（`success_rate = 1.000`），图3展示 Task 5 color 事件汇总与最终 JSON 写入确认。

```

Windows PowerShell
mathematical_logic/task_5 stage=original obs_variant=default map_variant=default seed=3 success=True steps=1084 reward=234.860
mathematical_logic/task_5 stage=original obs_variant=default map_variant=default seed=4 success=True steps=1084 reward=234.860
mathematical_logic/task_5 stage=original obs_variant=default map_variant=default seed=5 success=True steps=1084 reward=234.860
mathematical_logic/task_5 stage=spatial obs_variant=default map_variant=spatial_a seed=0 success=True steps=1077 reward=174.030
mathematical_logic/task_5 stage=spatial obs_variant=default map_variant=spatial_b seed=1 success=True steps=1173 reward=186.828
mathematical_logic/task_5 stage=spatial obs_variant=default map_variant=spatial_c seed=2 success=True steps=1127 reward=173.488
mathematical_logic/task_5 stage=color obs_variant=grayscale map_variant=default seed=0 success=True steps=1084 reward=234.860

mathematical_logic/task_1 [original]
episodes: 6
success_rate: 1.000
avg_steps: 270.0
avg_reward: 127.250
variants: {'default': 6}
map_variants: {'default': 6}
progress:
  key_collected: 1.000
  chest_opened: 1.000
  room_changed: 1.000
  door_opened: 1.000
  exit_reached: 1.000
  environment_completed: 1.000
  world_completed: 1.000

mathematical_logic/task_1 [spatial]
episodes: 3
success_rate: 1.000
avg_steps: 175.0
avg_reward: 128.133
variants: {'default': 3}
map_variants: {'spatial_a': 1, 'spatial_b': 1, 'spatial_c': 1}
progress:
  key_collected: 1.000
  chest_opened: 1.000
  room_changed: 1.000
  door_opened: 1.000
  exit_reached: 1.000
  environment_completed: 1.000
  world_completed: 1.000

mathematical_logic/task_1 [color]
episodes: 1
success_rate: 1.000
avg_steps: 270.0
avg_reward: 127.250
variants: {'grayscale': 1}
map_variants: {'default': 1}
progress:
  key_collected: 1.000
  chest_opened: 1.000
  room_changed: 1.000
  door_opened: 1.000

```

图 2: Task 1 统计汇总: original 6/6、spatial 3/3、color 1/1 均 success_rate = 1.000。

```

Windows PowerShell
world_completed: 1.000
progress:
  monster_killed: 1.000
  key_collected: 1.000
  chest_opened: 1.000
  gold_collected: 1.000
  agent_healed: 1.000
  button_pressed: 1.000
  room_changed: 1.000
  door_opened: 1.000
  exit_reached: 1.000
  environment_completed: 1.000
  world_completed: 1.000
game_event_totals:
  chest_opened: 4
  key_collected: 1
  gold_collected: 2
  item_collected: 0
  agent_healed: 1
  button_pressed: 1
  room_changed: 5
  door_opened: 1
  trap_triggered: 0
  monster_killed: 3
  exit_reached: 5
  environment_completed: 1
  world_completed: 1
Wrote JSON results to _final_check.json
(base) PS D:\Programming\Mathematical_Logic-nesylink\nesylink-mathlogic-project>

```

图 3: Task 5 color 事件汇总 (chest_opened/monster_killed/exit_reached 等) 与最终 Wrote JSON results to _final_check.json 写入确认。